

GSOC 2026 · MIT APP INVENTOR · CONTINUITY PHASE

ListView Cross-Platform Parity Audit

Findings report — Android vs. iOS behavioral comparison

BRANCH	feature/listview-parity-contract (base upstream/ucr)
AUDIT PERIOD	2026-08-21 - 2026-08-22
SCOPE	43 members exercised — 32 properties, 10 methods, 1 event
RESULT	8 findings — 2 affecting Android, 7 affecting iOS (one affects both)

1. Method

One test application, one identical sequence of steps, executed first on the Android emulator — captured as the reference — then on the iOS simulator, compared against it. Companions were built from the same commit on both platforms. Every member listed below was exercised on both platforms; no finding in this report is inferred from a single platform.

2. Summary of findings

Every member exercised in this audit, in declaration order. Rows marked  conformed on both platforms and are not discussed further — the table entry is the record. Rows marked  carry a section reference and are discussed below, in the order they appear here.

 Conforms to the contract	 Behaves differently — see reference	 Fix in review	- Not user-reachable
MEMBER	KIND	ANDROID	IOS
BackgroundColor	Property		
BounceEdgeEffect	Property		
DividerColor	Property		
DividerThickness	Property		 §3
ElementColor	Property		
ElementCornerRadius	Property		 §3
ElementMarginsWidth	Property		 §3
Elements	Property		
ElementsFromString	Property		
FontSize	Property		
FontSizeDetail	Property		
FontTypeface	Property		
FontTypefaceDetail	Property		
HintText	Property		 §4

ImageHeight	Property		§5
ImageWidth	Property		§5
ListData	Property		
ListViewLayout	Property		§6
Orientation	Property		§7
Selection	Property		
SelectionColor	Property		
SelectionDetailText	Property	§9	§9
SelectionIndex	Property		
ShowFilterBar	Property		
TextAlignmentDetail	Property		
TextAlignmentMain	Property		
TextColor	Property		
TextColorDetail	Property		
MultiSelect	Property		
SelectedItems	Property		
AddItem	Method		
AddItemAtIndex	Method		
AddItems	Method		
AddItemsAtIndex	Method		
CreateElement	Method		
GetMainText	Method		
GetDetailText	Method		
GetImageName	Method		
RemoveItemAtIndex	Method		
Refresh	Method	-	-
AfterPicking	Event		

Findings

Presented in the order the affected members appear in the summary table. Members not listed here conformed on both platforms.

3. Row chrome properties have no effect on iOS

Android  iOS 

DividerThickness · ElementCornerRadius · ElementMarginsWidth

EXPECTED

All three properties visibly affect row appearance.

ANDROID

DividerThickness renders a proportionally thick line, and 0 correctly hides the divider. ElementCornerRadius shows clearly rounded corners on the selected row. ElementMarginsWidth insets the rows and correctly hides the divider, as documented.

IOS

Tested at deliberately extreme values — thickness up to 50, radius up to 40, margins up to 20. None produced any visible change.

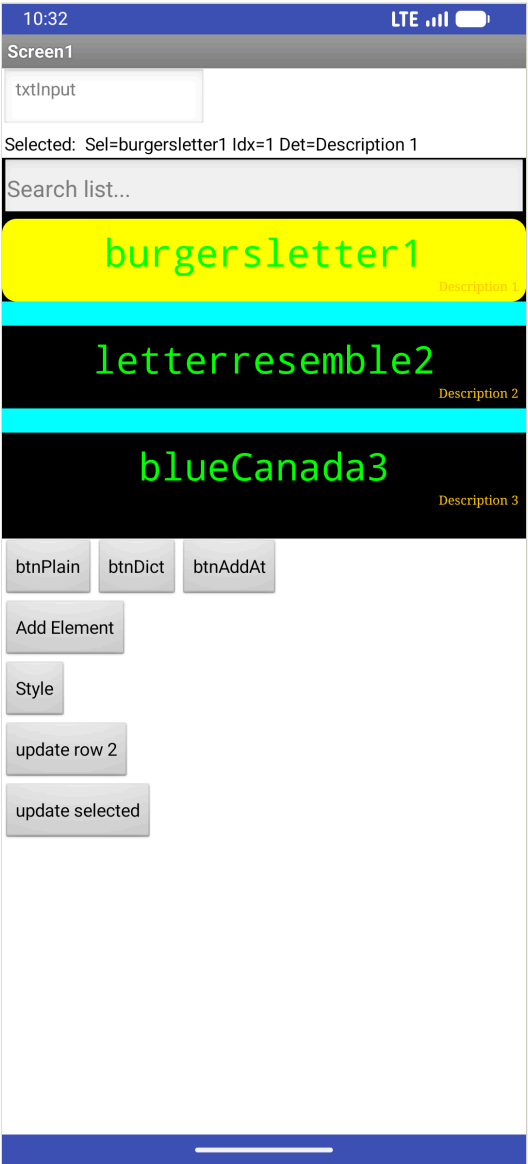


Fig. 3a – Android: thick divider, rounded selected corner

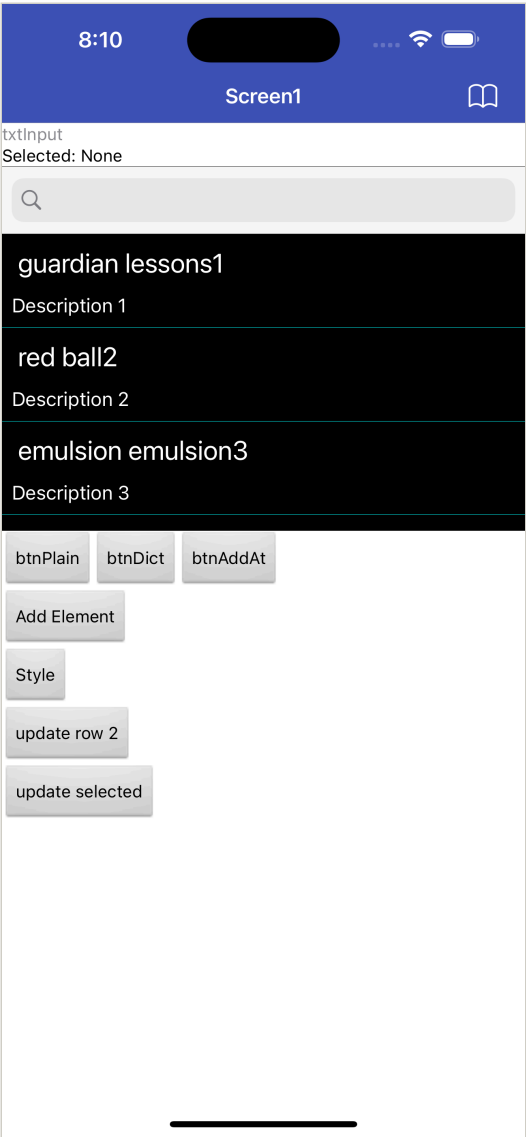


Fig. 3b – iOS: hairline divider, square corners

DividerThickness = 0 could not be meaningfully tested on iOS, since the property has no effect there to begin with.

4. Default filter-bar hint is never applied on iOS

Android  iOS 

HintText

EXPECTED

Both platforms declare the same default hint, "Search list...", and should display it without the app having to set anything.

ANDROID

Displays the Search list... placeholder. The constructor calls the setter explicitly (ListView.java:183), so the default reaches the widget.

IOS

Shows only a magnifier icon with no placeholder text.

Root cause. Not a platform convention — iOS search bars support placeholders, and the iOS setter is correctly wired. The problem is that the default never runs it:

```
// ListView.swift:672 – the setter works
@objc open var HintText: String {
    get { return _hint }
    set(hint) {
        _hint = hint
        filter.placeholder = _hint    // correct
    }
}

// ListView.swift:179 – but the default never invokes it
fileprivate var _hint = "Search list..." // stored default; setter not called
```

Assigning a stored property's default value does not invoke its setter, so `filter.placeholder` is never assigned at initialisation. The hint appears only if an app explicitly sets HintText from a block at runtime.

5. Images render at a quarter of the requested size on iOS

Android  iOS 

ImageWidth · ImageHeight

EXPECTED

The image renders at the requested pixel dimensions.

ANDROID

Rendered at the exact requested size (verified at 80×50).

IOS

Rendered at roughly one quarter of the requested size at every setting tested (80×50, 200×200, 400×400). In horizontal orientation the image is additionally stretched to the cell height, losing its aspect ratio.

Root cause. `ListView.swift` divides the requested size by four at three call sites (approx. lines 1005, 1057, 1106):

```
cell.imageView!.widthAnchor.constraint(equalToConstant: CGFloat(_imageWidth / 4)),
cell.imageView!.heightAnchor.constraint(equalToConstant: CGFloat(_imageHeight / 4))
```

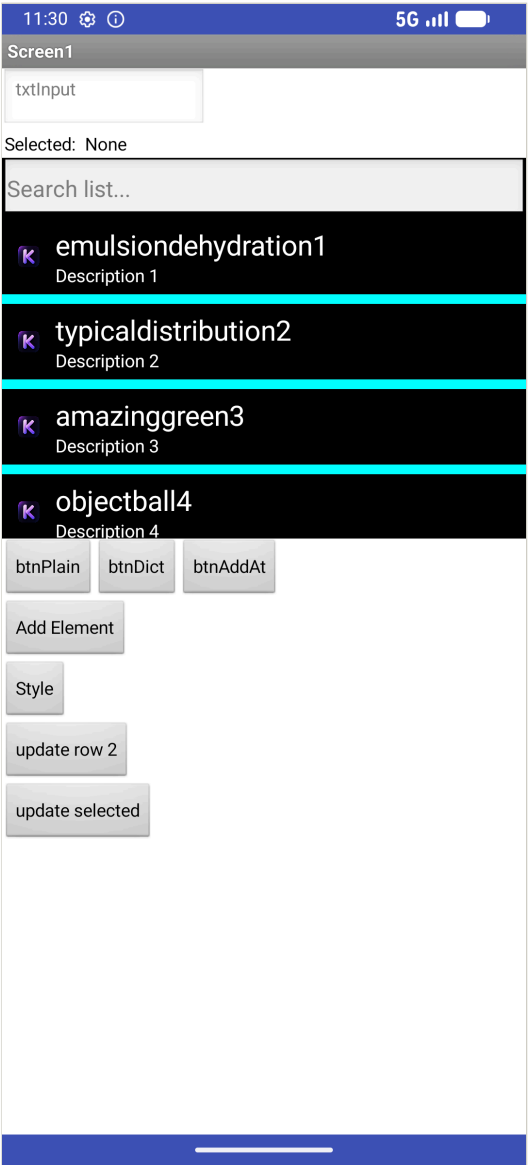


Fig. 5a – Android: 80×50 requested and rendered

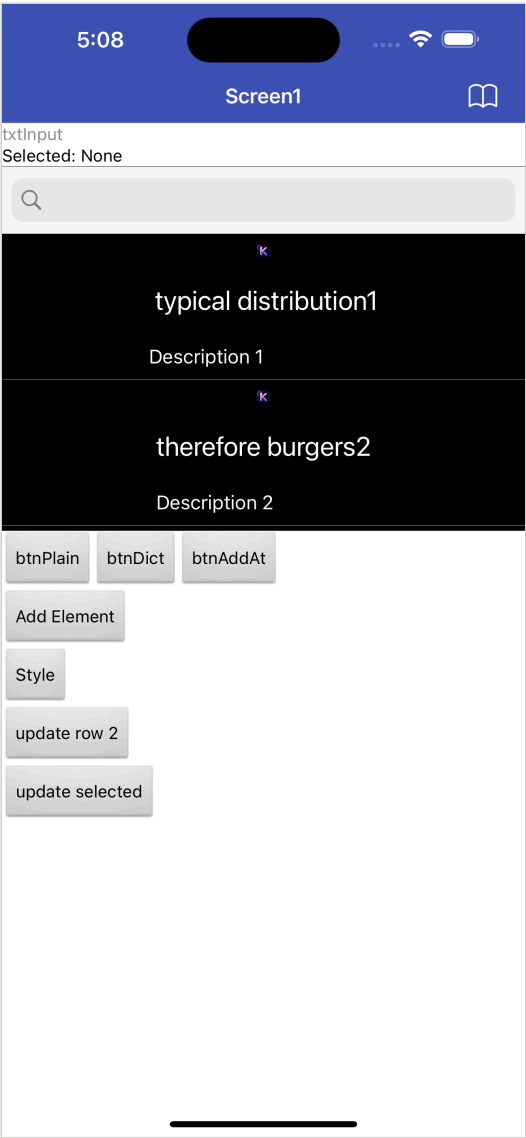


Fig. 5b – iOS: same request, rendered ~¼ size

6. Content is dropped at large font sizes on iOS

 Android  iOS 

ListViewLayout

EXPECTED

A row grows to accommodate wrapped main text together with its detail line.

ANDROID

Main text wraps across two lines and the detail line renders in full — the row grows and nothing is lost.

IOS

At the identical setting (image + main text + detail layout, `FontSize` raised to force wrapping): one row rendered only the first word of the main text with the detail line absent entirely; another rendered no text at all.

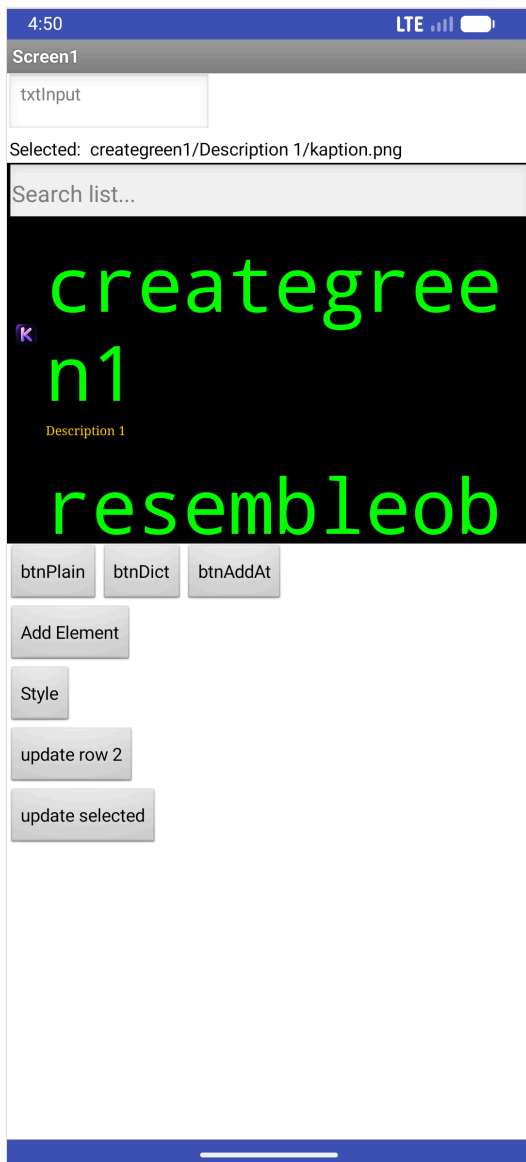


Fig. 6a – Android: row grows, full content renders

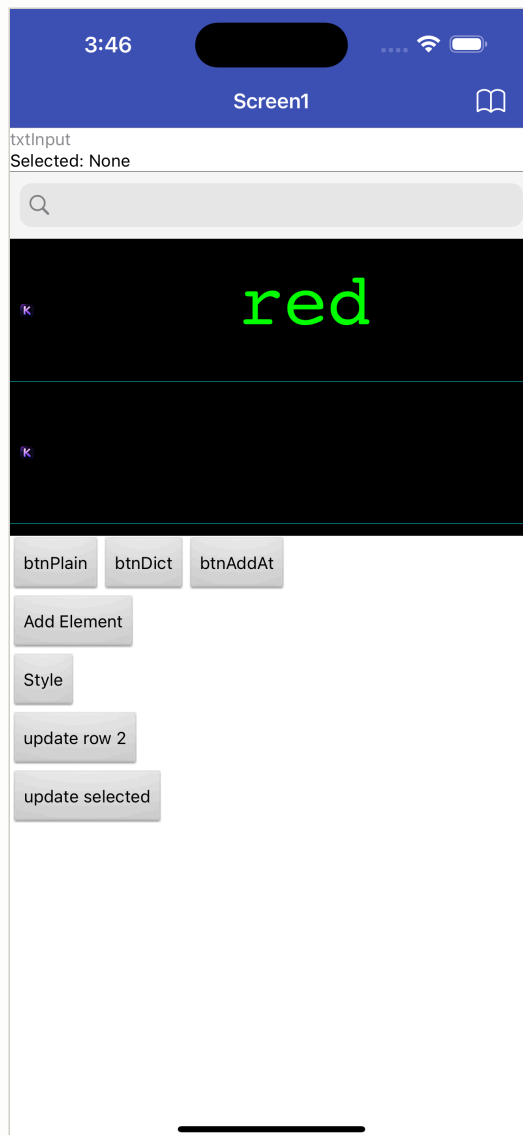


Fig. 6b – iOS: first word only, detail line missing

All six layouts were otherwise verified structurally identical on both platforms, including the two-text horizontal 50/50 split from PR #3944, which still holds. This defect is likely related to the same fixed-row-height assumption behind §3 and §5 – recommend investigating them together.

7. Horizontal orientation differs in three ways on iOS

Android  iOS 

Orientation

EXPECTED

Documented behaviour: one element visible at a time, swipe left or right to browse.

ANDROID

One element at a time, as documented. Long text renders in full. The filter bar remains visible and functional.

IOS

Multiple elements visible in a continuous scroll; long text truncated with an ellipsis; the filter bar absent entirely, with the list beginning immediately below the status area. All three reproduced in two separate layouts, so they are general to horizontal orientation rather than one layout’s quirk.

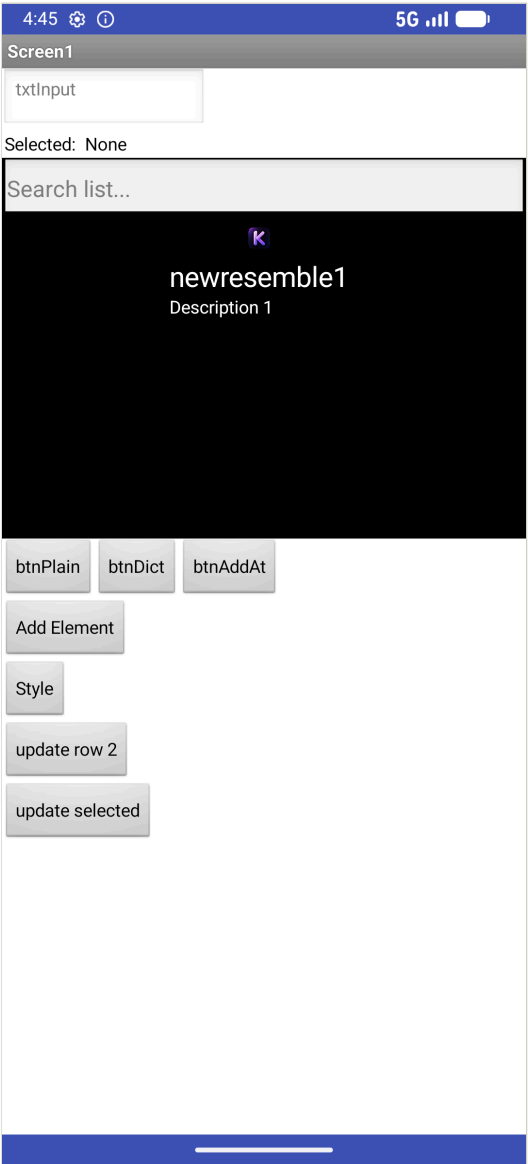


Fig. 7a – Android: one element, filter bar present

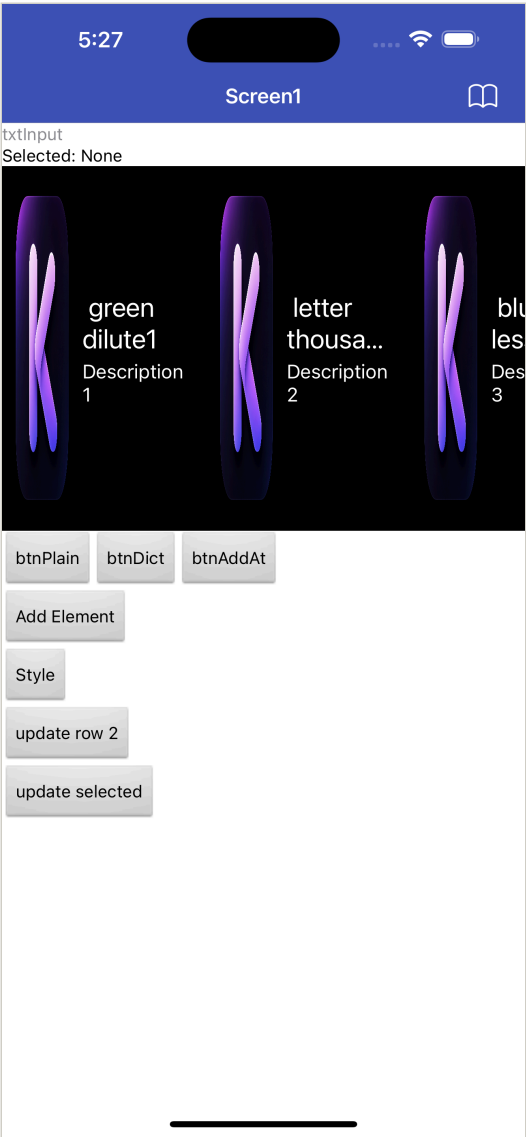


Fig. 7b – iOS: multiple elements, no filter bar, truncated

Horizontal orientation uses a separate `UICollectionView` code path on iOS, distinct from the vertical table — likely why none of this affects vertical mode.

9. SelectionDetailText behaves inconsistently on both platforms

Android  iOS 

SelectionDetailText

EXPECTED

The property reflects the currently selected row at all times, and is empty when nothing is selected. Both platforms report the same value for the same state.

ANDROID

Select a row on a plain-text list, where no row carries detail text at all: `Selection` and `SelectionIndex` read correctly, but `SelectionDetailText` returns detail text belonging to a dictionary list that was already replaced. The cause is that the field is assigned only when the selected item is a dictionary carrying a main-text key — every other path leaves the previous value untouched. Also reachable by clearing the selection, and by setting an out-of-range index.

IOS

Correctly empty in all three of the above cases. However, before any row is ever selected, iOS defaults the field to an empty string (`ListView.swift:162`) while Android defaults to the literal string `"Uninitialized"` (`ListView.java:95`) — so a fresh app reading this property before any selection sees different values on each platform.

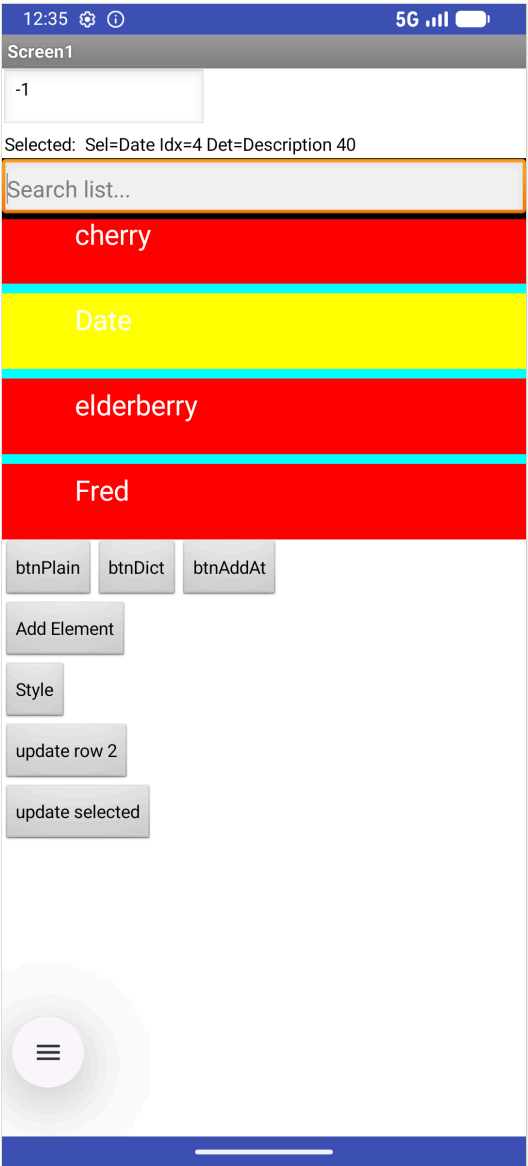


Fig. 9a – Android: plain-text list (no row has detail text), yet Det="Description 40"

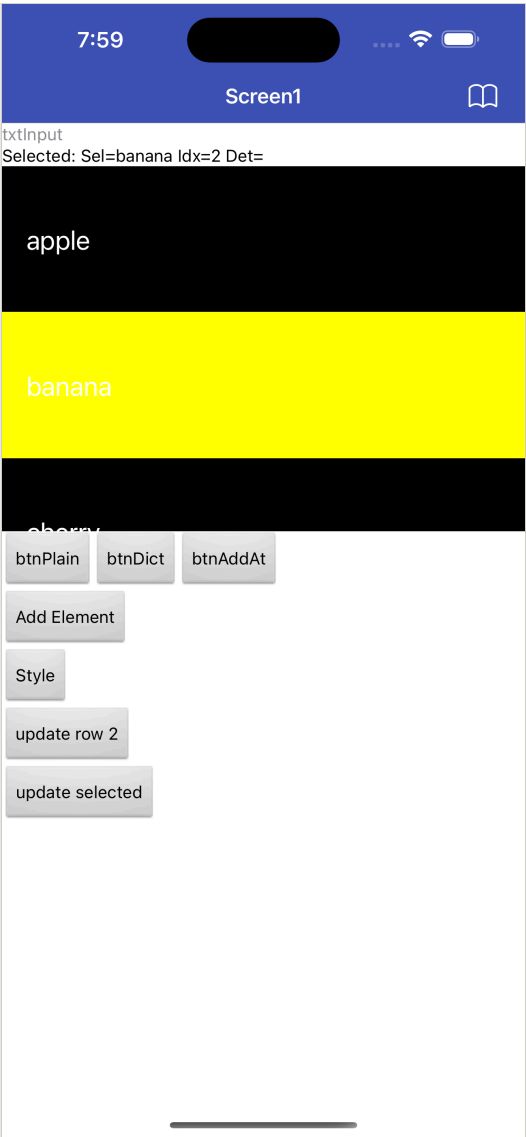


Fig. 9b – iOS: same plain-text scenario, Det correctly empty

Both figures show the identical scenario: a plain-text list in which no row carries detail text, with a row selected. iOS returns empty, as it should. Android returns detail text belonging to a dictionary list that had already been replaced.

ListView Cross-Platform Parity Audit

Member-by-member detail: LISTVIEW_PARITY_CONTRACT.md